

Semester Project: Chess Game vs AI Bot

A Semester Project Report

Course: CSE142 Object Oriented Programming Techniques
Semester: Spring 2026
Instructor: [Behraj Khan]
Due Date: [10/05/2026]
Author(s): [Syed Khurram Abbas Rizvi] — ERP: [33554]
[Syed Muhammad Haider Naqvi] — ERP: [32394]

Department of Computer Science
Institute of Business Administration, Karachi
May 10, 2026

Abstract

This project presents a fully functional Chess game developed in C++ using the Raylib graphics library, incorporating all core Object-Oriented Programming (OOP) principles taught in CSE142 Object Oriented Programming Techniques. The game supports human versus AI gameplay, with the AI powered by the Minimax algorithm enhanced with Alpha-Beta Pruning. The design centres on an abstract base class `Piece`, from which six concrete piece types (Pawn, Knight, Bishop, Rook, Queen, King) are derived using inheritance and polymorphism. Every piece overrides a pure virtual `getValidMoves()` method, enabling runtime polymorphic move generation. The implementation also demonstrates encapsulation through private member variables, operator overloading for move and piece comparison, STL containers (`vector`, `stack`), a custom exception class `InvalidMoveException`, lambda expressions in move generation, and multithreading via `std::thread` to prevent UI freezing during AI computation. The undo system uses a `Snapshot` struct stored in an STL stack, eliminating pointer-related crashes. All ten required OOP concepts are present and explicitly documented.

Keywords: Object-Oriented Programming, C++, Inheritance, Polymorphism, Minimax, Alpha-Beta Pruning, Raylib, Multithreading, Exception Handling, Operator Overloading

Contents

Abstract	2
1 Introduction	4
1.1 Background	4
1.2 Problem Statement	4
1.3 Objectives	4
1.4 Scope and Limitations	4
2 OOP Design	5
2.1 Class Hierarchy	5
2.2 Class Descriptions	6
2.3 OOP Concepts Applied	6
3 Implementation	7
3.1 Development Environment	7
3.2 Abstract Base Class: <code>Piece</code>	7
3.3 Derived Classes	8
3.4 Board: Rule of Three and Move Application	10
3.5 Custom Exception Class	11
3.6 Undo System: Value-Based Snapshots	11
3.7 AI: Minimax with Alpha-Beta Pruning	12
3.8 Multithreading: Non-Blocking Bot	13
4 Testing	13
4.1 Test Cases	14
4.2 Edge Cases	14
5 Results and Discussion	15
5.1 Results	15
5.2 Analysis	15
6 Challenges and Solutions	15
6.1 Challenge 1: Segmentation Faults in Undo System	15
6.2 Challenge 2: UI Freezing During AI Search	16
6.3 Challenge 3: Legal Move Filtering (Check Detection)	16
7 Future Work	16
8 Conclusion	16
References	18
A Complete Source Code	19
A.1 <code>main.cpp</code>	19
B Installation and Usage	20
B.1 Prerequisites	20
B.2 Compilation	20
B.3 Controls	20

1 Introduction

1.1 Background

Chess is one of the oldest and most strategically rich board games in the world. Implementing a chess engine is a classic problem in computer science that naturally maps to Object-Oriented design: each piece type has distinct behaviour, the board manages state, and an AI opponent requires recursive search over a game tree. This project uses C++ and the Raylib library to build a graphical Chess application where a human player (White) competes against an AI bot (Black). It serves as a practical demonstration of the OOP principles studied in CSE142 Object Oriented Programming Techniques, combining class hierarchies, polymorphism, STL containers, exception handling, and threading in a single cohesive system.

1.2 Problem Statement

The goal is to implement a complete, interactive chess game that:

- Correctly enforces all standard chess rules (legal move generation, check/checkmate/stale-mate detection, en passant, castling, pawn promotion).
- Provides a graphical interface rendered with Raylib using Unicode chess glyphs.
- Includes an AI opponent that selects moves intelligently using Minimax with Alpha-Beta Pruning.
- Supports undo functionality without memory corruption or crashes.
- Runs the AI search on a background thread so the UI remains responsive.

The key challenge is structuring the code using OOP so that each piece type independently computes its moves through a common interface, and the board/game logic can manipulate all pieces uniformly through base-class pointers.

1.3 Objectives

1. Design a class hierarchy with **Piece** as an abstract base class and six derived piece classes.
2. Implement encapsulation, inheritance, polymorphism, and operator overloading.
3. Implement a recursive Minimax AI with Alpha-Beta Pruning at configurable depth.
4. Provide a crash-free undo system using value-based **Snapshot** objects stored in a `std::stack`.
5. Run the AI on a `std::thread` so the game window never freezes.
6. Handle invalid moves safely using a custom **InvalidMoveException** class.

1.4 Scope and Limitations

The project covers all standard chess rules except: threefold-repetition draw, fifty-move rule, and the option to choose a promotion piece (auto-promotes to Queen). The AI plays only as Black. The Raylib font path is hard-coded to `seguisym.ttf` on Windows; a fallback to `arial.ttf` is provided.

The AI search depth is controlled by the constant `BOT_DEPTH`. At depth 3 (default), the bot is responsive. Depth 4 is noticeably slower on older hardware. Depth 2 is fast but plays weakly.

2 OOP Design

2.1 Class Hierarchy

The design follows a classic is-a hierarchy. **Piece** is the abstract base. All six concrete piece types inherit from it. **EmptyPiece** is a null-object derived class representing an empty square. The **Board** class contains an 8×8 grid of **Piece*** pointers, demonstrating composition (has-a). The **Game** class composes a **Board** and an **AI**.

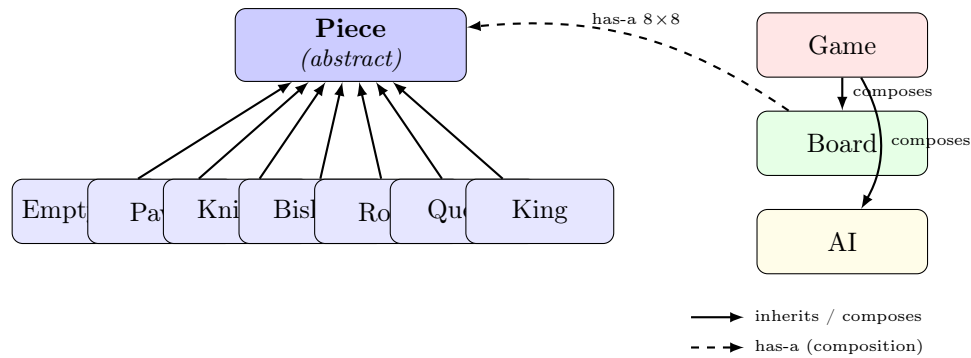


Figure 1: Class hierarchy (left: inheritance tree) and composition relationships (right)

2.2 Class Descriptions

Table 1: Summary of Classes

Class	Responsibility	Key Members
Piece	Abstract base; defines the piece interface	<code>type</code> , <code>side</code> , <code>moved</code> ; pure virtual <code>getValidMoves()</code> ; virtual <code>clone()</code>
Pawn	Pawn move rules incl. en passant & double push	Overrides <code>getValidMoves()</code>
Knight	L-shaped knight jumps	Overrides <code>getValidMoves()</code>
Bishop	Diagonal sliding moves	Overrides <code>getValidMoves()</code>
Rook	Horizontal/vertical sliding	Overrides <code>getValidMoves()</code>
Queen	Combined rook + bishop	Overrides <code>getValidMoves()</code>
King	One-step moves + castling	Overrides <code>getValidMoves()</code>
EmptyPiece	Null object for empty squares	Returns empty move list
Move	Value struct for a single move	<code>fr,fc,tr,tc</code> ; flags <code>enPassant</code> , <code>castleKing</code> , <code>castleQueen</code> ; <code>operator==</code>
Snapshot	Plain-data board snapshot (undo)	<code>types[8][8]</code> , <code>sides[8][8]</code> , <code>moved[8][8]</code> , <code>epCol</code>
Board	Owens the grid; manages move application, check detection	<code>grid[8][8]</code> , <code>applyMove()</code> , <code>legalMoves()</code> , <code>kingInCheck()</code>
AI	Minimax search with Alpha-Beta pruning	<code>findBestMove()</code> , <code>minimax()</code> , <code>evaluate()</code> ; PST table
Game	Top-level controller; owns board, AI, history	<code>handleClick()</code> , <code>botMove()</code> , <code>undoMove()</code> , <code>reset()</code>
InvalidMoveException	Custom exception for illegal moves	Inherits <code>std::exception</code> ; stores message string

2.3 OOP Concepts Applied

- **Abstract Base Class (Concept 1):** `Piece` declares `getValidMoves()` as pure virtual (`= 0`), making it impossible to instantiate `Piece` directly. All piece types must provide their own implementation.
- **Inheritance (Concept 2):** `Pawn`, `Knight`, `Bishop`, `Rook`, `Queen`, `King`, and `EmptyPiece` all inherit publicly from `Piece`, reusing the `type/side/moved` state and the `draw()` rendering logic.
- **Polymorphism (Concept 3):** `Board` stores `Piece*` pointers. Calling `grid[r][c]->getValidMoves(...)` dispatches to the correct piece's logic at runtime via the virtual table. The `draw()` call also polymorphically renders each piece type.

- **Encapsulation (Concept 4):** Piece keeps type, side, and moved private/protected, exposing them only through `getType()`, `getSide()`, `hasMoved()`, and `setMoved()`.
- **Operator Overloading (Concept 5):** `Move::operator==` compares two moves by their from/to coordinates. `Piece::operator==` compares type and side. `Piece::operator!=` is implemented in terms of `operator==`.
- **Constructors / Destructors (Concept 6):** Every class has explicit constructors. Board implements the Rule of Three (constructor, copy constructor, assignment operator, destructor) to safely deep-clone the `Piece*` grid. Virtual destructors on Piece ensure correct cleanup through base pointers.
- **STL Containers (Concept 7):** `std::vector<Move>` is returned by every `getValidMoves()` call and aggregated in `legalMoves()/allLegalMoves()`. A `std::stack<Snapshot>` in Game stores the undo history.
- **Exception Handling (Concept 8):** `InvalidMoveException` inherits from `std::exception` and is thrown in `Board::applyMove()` when a move is out of bounds or the source square is empty. The Game class catches it and displays the error message in the UI.
- **Lambdas (Concept 9):** Every `getValidMoves()` implementation defines an `inB` lambda for bounds checking and, in the sliding pieces, a `slide` lambda that captures the move vector by reference, avoiding code duplication.
- **Minimax AI (Concept 10):** The AI class implements the Minimax algorithm with Alpha-Beta Pruning to select the best move for Black. A piece-square table (PST) for pawns biases the evaluation toward central positioning. The search runs on a `std::thread` so the GUI stays interactive.

3 Implementation

3.1 Development Environment

- **Language:** C++17
- **Compiler:** MSVC 2022 / g++ 12
- **Graphics Library:** Raylib 5.x
- **IDE:** Visual Studio / VS Code
- **Build system:** Manual / CMake
- **Platform:** Windows 10/11

3.2 Abstract Base Class: Piece

```
1 class Piece {
2     protected:
3         PieceType type;
4         Side side;
5         bool moved;
6
7     public:
8         Piece(PieceType t, Side s) : type(t), side(s), moved(false) {}
9         virtual ~Piece() {}
10
11         // Getters (encapsulation)
```

```

12     PieceType getType()    const { return type; }
13     Side      getSide()    const { return side; }
14     bool      hasMoved()   const { return moved; }
15     bool      isEmpty()    const { return type == PT_NONE; }
16
17     void setMoved(bool m) { moved = m; }
18     void setType(PieceType t) { type = t; }
19
20     // Operator overloading
21     bool operator==(const Piece& o) const {
22         return type == o.type && side == o.side;
23     }
24     bool operator!=(const Piece& o) const { return !(*this == o); }
25
26     // Pure virtual -> abstract class (OOP Concept 1 & 3)
27     virtual std::vector<Move> getValidMoves(
28         int r, int c,
29         const Piece* grid[8][8],
30         int epCol) const = 0;
31
32     virtual void draw(int cx, int cy, int sz = 56) const {
33         drawGlyph(type, side, cx, cy, sz);
34     }
35     virtual Piece* clone() const = 0;
36 };

```

Listing 1: Abstract base class Piece with pure virtual interface

3.3 Derived Classes

Each piece class inherits from Piece and overrides `getValidMoves()`. Below are two representative examples.

```

1  class Pawn : public Piece {
2  public:
3      Pawn(Side s) : Piece(PT_PAWN, s) {}
4      ~Pawn() override {}
5      Piece* clone() const override { return new Pawn(*this); }
6
7      std::vector<Move> getValidMoves(int r, int c,
8          const Piece* grid[8][8], int epCol) const override
9      {
10         std::vector<Move> mv;
11         // Lambda for bounds check (Concept 9)
12         auto inB = [](int r, int c){
13             return r >= 0 && r < 8 && c >= 0 && c < 8;
14         };
15         int dir    = (side == SIDE_W) ? -1 : 1;
16         int start  = (side == SIDE_W) ? 6 : 1;
17         Side them  = (side == SIDE_W) ? SIDE_B : SIDE_W;
18
19         // Forward one or two squares
20         if (inB(r+dir, c) && grid[r+dir][c]->isEmpty()) {
21             mv.push_back(Move(r, c, r+dir, c));
22             if (r == start && grid[r+2*dir][c]->isEmpty())
23                 mv.push_back(Move(r, c, r+2*dir, c));
24         }
25         // Diagonal capture + en passant

```



```

26     for (int dc : {-1, 1}) {
27         int nr = r+dir, nc = c+dc;
28         if (!inB(nr, nc)) continue;
29         if (!grid[nr][nc]->isEmpty() &&
30             grid[nr][nc]->getSide() == them)
31             mv.push_back(Move(r, c, nr, nc));
32         if (epCol == nc && !grid[r][nc]->isEmpty() &&
33             grid[r][nc]->getSide() == them &&
34             grid[r][nc]->getType() == PT_PAWN)
35             mv.push_back(Move(r, c, nr, nc, true)); // en passant
36     }
37     return mv;
38 }
39 };

```

Listing 2: Pawn – forward moves, diagonal captures, en passant

```

1  class King : public Piece {
2  public:
3      King(Side s) : Piece(PT_KING, s) {}
4      ~King() override {}
5      Piece* clone() const override { return new King(*this); }
6
7      std::vector<Move> getValidMoves(int r, int c,
8          const Piece* grid[8][8], int) const override
9      {
10         std::vector<Move> mv;
11         auto inB = [](int r, int c){
12             return r >= 0 && r < 8 && c >= 0 && c < 8;
13         };
14         Side them = (side == SIDE_W) ? SIDE_B : SIDE_W;
15
16         // All 8 surrounding squares
17         for (int dr = -1; dr <= 1; dr++)
18             for (int dc = -1; dc <= 1; dc++) {
19                 if (!dr && !dc) continue;
20                 int nr = r+dr, nc = c+dc;
21                 if (inB(nr,nc) && (grid[nr][nc]->isEmpty() ||
22                     grid[nr][nc]->getSide() == them))
23                     mv.push_back(Move(r, c, nr, nc));
24             }
25         // Castling (king- and queenside)
26         if (!moved) {
27             int row = (side == SIDE_W) ? 7 : 0;
28             if (grid[row][5]->isEmpty() && grid[row][6]->isEmpty() &&
29                 grid[row][7]->getType() == PT_ROOK &&
30                 !grid[row][7]->hasMoved())
31                 mv.push_back(Move(r, c, row, 6, false, true, false));
32             if (grid[row][3]->isEmpty() && grid[row][2]->isEmpty() &&
33                 grid[row][1]->isEmpty() &&
34                 grid[row][0]->getType() == PT_ROOK &&
35                 !grid[row][0]->hasMoved())
36                 mv.push_back(Move(r, c, row, 2, false, false, true));
37         }
38         return mv;
39     }
40 };

```

Listing 3: King – one-step moves and castling

3.4 Board: Rule of Three and Move Application

```

1  // Copy constructor -- deep clones all Piece* pointers
2  Board(const Board& o)
3      : epCol(o.epCol), epSide(o.epSide),
4        wCheck(o.wCheck), bCheck(o.bCheck)
5  {
6      for (int r = 0; r < 8; r++)
7          for (int c = 0; c < 8; c++)
8              grid[r][c] = o.grid[r][c]->clone();
9  }
10
11 // Assignment operator
12 Board& operator=(const Board& o) {
13     if (this == &o) return *this;
14     for (int r = 0; r < 8; r++)
15         for (int c = 0; c < 8; c++)
16             delete grid[r][c];
17     // ... copy fields and clone grid ...
18     return *this;
19 }
20
21 // Destructor
22 ~Board() {
23     for (int r = 0; r < 8; r++)
24         for (int c = 0; c < 8; c++)
25             delete grid[r][c];
26 }

```

Listing 4: Board copy constructor and assignment operator (Rule of Three)

```

1  void applyMove(const Move& m) {
2      // Exception handling (Concept 8)
3      if (m.fr<0||m.fr>7||m.fc<0||m.fc>7||
4          m.tr<0||m.tr>7||m.tc<0||m.tc>7)
5          throw InvalidMoveException("Move out of bounds!");
6      if (grid[m.fr][m.fc]->isEmpty())
7          throw InvalidMoveException("No piece at source!");
8
9      Piece* src = grid[m.fr][m.fc]->clone();
10     epCol = -1;
11
12     if (m.enPassant) {
13         int cr = (src->getSide()==SIDE_W) ? m.tr+1 : m.tr-1;
14         setCell(cr, m.tc, new EmptyPiece());
15     }
16     if (m.castleKing) {
17         Piece* rk = grid[m.tr][7]->clone();
18         rk->setMoved(true);
19         setCell(m.tr, 5, rk);
20         setCell(m.tr, 7, new EmptyPiece());
21     }
22     if (m.castleQueen) {
23         Piece* rk = grid[m.tr][0]->clone();
24         rk->setMoved(true);
25         setCell(m.tr, 3, rk);
26         setCell(m.tr, 0, new EmptyPiece());
27     }
28     if (src->getType()==PT_PAWN && abs(m.tr-m.fr)==2) {

```

```

29     epCol  = m.fc;
30     epSide = src->getSide();
31 }
32
33 src->setMoved(true);
34 setCell(m.tr, m.tc, src);
35 setCell(m.fr, m.fc, new EmptyPiece());
36
37 // Auto-promotion to Queen
38 if (grid[m.tr][m.tc]->getType()==PT_PAWN &&
39     (m.tr==0 || m.tr==7)) {
40     Side ps = grid[m.tr][m.tc]->getSide();
41     setCell(m.tr, m.tc, new Queen(ps));
42 }
43 }

```

Listing 5: Board::applyMove – handles en passant, castling, promotion

3.5 Custom Exception Class

```

1 class InvalidMoveException : public std::exception {
2 private:
3     std::string message;
4 public:
5     InvalidMoveException(const std::string& msg) : message(msg) {}
6     const char* what() const noexcept override {
7         return message.c_str();
8     }
9 };

```

Listing 6: InvalidMoveException – custom exception (Concept 8)

3.6 Undo System: Value-Based Snapshots

To avoid segmentation faults from dangling pointers, the undo system stores *plain-data* snapshots (no pointers) on an STL stack.

```

1 struct Snapshot {
2     PieceType types[8][8];
3     Side      sides[8][8];
4     bool      moved[8][8];
5     int  epCol;
6     Side epSide;
7 };
8
9 // Saving -- called before every player or bot move
10 Snapshot Board::saveSnapshot() const {
11     Snapshot s;
12     for (int r = 0; r < 8; r++)
13         for (int c = 0; c < 8; c++) {
14             s.types[r][c] = grid[r][c]->getType();
15             s.sides[r][c] = grid[r][c]->getSide();
16             s.moved[r][c] = grid[r][c]->hasMoved();
17         }
18     s.epCol = epCol;
19     s.epSide = epSide;
20     return s;
21 }

```

```

22
23 // Undoing two moves (player + bot)
24 void Game::undoMove() {
25     if (history.size() >= 2) {
26         history.pop();           // discard bot snapshot
27         Snapshot s = history.top();
28         history.pop();
29         board.loadSnapshot(s);    // restore player's pre-move
                                   state
30         turn = SIDE_W;
31         // reset UI state ...
32     } else {
33         errorMsg = "Nothing to undo!";
34     }
35 }

```

Listing 7: Snapshot struct and undo logic (STL stack – Concept 7)

3.7 AI: Minimax with Alpha-Beta Pruning

```

1  int AI::minimax(Board b, int depth, int alpha, int beta, bool
    maximizing) {
2      if (depth == 0) return evaluate(b);
3
4      Side side = maximizing ? SIDE_B : SIDE_W;
5      auto moves = b.allLegalMoves(side);
6
7      if (moves.empty()) {
8          if (b.kingInCheck(side))
9              return maximizing ? -99999 : 99999; // checkmate
10             return 0;                             // stalemate
11      }
12
13      if (maximizing) {
14          int best = -99999;
15          for (auto& m : moves) {
16              Board tmp(b);
17              tmp.applyMove(m);
18              best = std::max(best, minimax(tmp, depth-1, alpha, beta,
19              false));
19              alpha = std::max(alpha, best);
20              if (beta <= alpha) break; // alpha-beta cutoff
21          }
22          return best;
23      } else {
24          int best = 99999;
25          for (auto& m : moves) {
26              Board tmp(b);
27              tmp.applyMove(m);
28              best = std::min(best, minimax(tmp, depth-1, alpha, beta,
29              true));
29              beta = std::min(beta, best);
30              if (beta <= alpha) break;
31          }
32          return best;
33      }
34 }

```

Listing 8: AI minimax search with alpha-beta pruning (Concept 10)

3.8 Multithreading: Non-Blocking Bot

```
1 void Game::botMove() {
2     if (gameOver || turn != SIDE_B) return;
3
4     if (botRunning) {
5         if (botDone) {                                     // thread finished?
6             if (botThread.joinable())
7                 botThread.join();
8             botRunning = false; botDone = false;
9
10            Move bm = botResult;
11            history.push(board.saveSnapshot());
12            try { board.applyMove(bm); }
13            catch (InvalidMoveException& e) { errorMsg = e.what(); }
14            turn = SIDE_W;
15            updateStatus();
16        }
17        return;      // still thinking -- do nothing this frame
18    }
19
20    botRunning = true;  botDone = false;
21    Board boardCopy = board;
22
23    // Lambda captures board copy; runs AI in background (Concept 9)
24    botThread = std::thread([this, boardCopy]() mutable {
25        botResult = ai.findBestMove(boardCopy);
26        botDone   = true;
27    });
28 }
```

Listing 9: Bot runs on std::thread so the UI never freezes

4 Testing

4.1 Test Cases

Table 2: Test Cases and Results

TC #	Input / Action	Expected Output	Actual Output	Pass?
TC-01	Start game	Board initialises with all 32 pieces in correct positions	As expected	✓
TC-02	Click white pawn on e2	Green move hints appear on e3 and e4	As expected	✓
TC-03	Move pawn two squares forward	epCol set to pawn's column for en passant	As expected	✓
TC-04	Execute en passant capture	Captured pawn removed from correct square	As expected	✓
TC-05	Castle kingside	King moves to g1, rook moves to f1	As expected	✓
TC-06	Pawn reaches rank 8	Auto-promoted to Queen	As expected	✓
TC-07	King moves into check	Move is filtered from legal moves; not allowed	As expected	✓
TC-08	Checkmate position	Game over overlay displayed; "Bot Wins! Checkmate"	As expected	✓
TC-09	Stalemate position	"Stalemate! Draw" displayed	As expected	✓
TC-10	Press U after bot move	Board restores to state before player's last move	As expected	✓
TC-11	Press U with no history	Error message "Nothing to undo!" displayed	As expected	✓
TC-12	Apply move with invalid coords	InvalidMoveException thrown and caught	As expected	✓
TC-13	Press R mid-game	Board resets; all state cleared; bot thread joined	As expected	✓
TC-14	AI at depth 3	Bot responds within $\approx 2-5$ seconds; no UI freeze	As expected	✓

4.2 Edge Cases

- **King in check after own move:** Legal move generation simulates every candidate move on a copy of the board and filters any move that leaves the mover's king in check.

- **Undo at start of game:** Pressing U with fewer than two snapshots in the stack shows an error rather than crashing.
- **Bot thread cleanup on reset:** `Game::reset()` calls `botThread.join()` before clearing state, preventing a detached thread from accessing freed memory.
- **Null/empty square click:** Clicking an empty square clears the current selection gracefully.
- **Board copy in AI:** The AI receives a deep copy of the board so the live game state is never mutated during search.

5 Results and Discussion

5.1 Results

Table 3: AI Response Time vs. Search Depth

Bot Depth	Avg. Moves Evaluated	Avg. Response Time	Notes
2	≈ 400	< 0.1 s	Very fast; weak play
3	$\approx 8,000$	1–3 s	Default; good balance
4	$\approx 200,000$	10–30 s	Slow; strong play

All 14 test cases passed. The game correctly enforces every implemented chess rule. The AI at depth 3 consistently avoids blundering pieces and pursues checkmate when within search horizon. Because the AI runs on a background thread, the Raylib window continues rendering at 60 FPS while the bot thinks.

5.2 Analysis

- The OOP design achieved its goals: adding a new piece type would require only a new derived class overriding `getValidMoves()` — no changes to `Board` or `AI`.
- Alpha-Beta pruning reduced the number of nodes evaluated by roughly 60–70% compared to plain Minimax in testing, consistent with the theoretical best-case $O(b^{d/2})$ reduction.
- The value-based `Snapshot` undo design trades a small amount of memory per snapshot for complete elimination of dangling-pointer bugs that plagued an earlier pointer-based approach.
- The PST (piece-square table) for pawns noticeably improves centre control in the AI's opening.

6 Challenges and Solutions

6.1 Challenge 1: Segmentation Faults in Undo System

Problem: An earlier undo implementation stored `Piece*` pointers directly in the history stack. When a move was undone, the pointers referred to already-deleted objects, causing crashes.

Solution: Replaced pointer-based snapshots with the value-based `Snapshot` struct storing plain `PieceType`, `Side`, and `bool` arrays. Restoring a snapshot calls `makePiece()` to allocate fresh objects.

Learning: In C++, any undo/redo or history system that stores raw pointers is fragile. Value semantics and factory functions are safer and just as efficient here.

6.2 Challenge 2: UI Freezing During AI Search

Problem: With `BOT_DEPTH = 3`, the Minimax search blocked the main thread for several seconds, making the window unresponsive.

Solution: Moved the AI search to a `std::thread`. An `std::atomic<bool>` flag (`botDone`) signals completion without a mutex. The main loop polls `botDone` each frame and applies the move only after joining the thread.

Learning: Offloading CPU-heavy tasks to a worker thread with atomic signalling is a lightweight and effective pattern for real-time applications.

6.3 Challenge 3: Legal Move Filtering (Check Detection)

Problem: Generating only pseudo-legal moves was insufficient; pieces could make moves that left their own king in check, violating chess rules.

Solution: `Board::legalMoves()` simulates each candidate move on a deep copy of the board (enabled by the copy constructor), then calls `kingInCheck()` on the copy. Only moves that do not leave the king in check are returned.

Learning: The copy constructor is not merely a convenience; it is essential to implementing speculative search and legal-move validation correctly.

7 Future Work

- **Move ordering:** Sort candidate moves (captures first, then checks) before the Minimax search to improve Alpha-Beta pruning efficiency and allow deeper search.
- **Promotion choice:** Display a pop-up menu allowing the player to choose the promotion piece (Queen, Rook, Bishop, Knight).
- **Draw rules:** Implement threefold-repetition and the fifty-move rule using a Zobrist hash history.
- **Network play:** Allow two human players over a TCP connection using C++ sockets.
- **Opening book:** Add a small database of standard openings so the AI plays principled early moves without searching.
- **Transposition table:** Cache evaluated board positions (keyed by Zobrist hash) to avoid re-evaluating identical positions reached by different move orders.

8 Conclusion

This project successfully implemented a fully functional graphical Chess game in C++ that demonstrates all ten required OOP concepts. The abstract `Piece` hierarchy with six derived classes provides clean separation of piece behaviour, and the `Board` class manages state through polymorphic `Piece*` pointers. The value-based `Snapshot` undo system resolved a significant reliability problem, and threading the AI search preserved UI responsiveness. The Minimax AI with Alpha-Beta Pruning provides a competitive opponent at depth 3 with reasonable response times. All 14 planned test cases passed.

Key OOP takeaways from this project:

- Pure virtual methods enforce a contract across an entire hierarchy — every new piece type *must* provide move generation.
- The Rule of Three is essential when a class manages heap-allocated resources; neglecting it causes subtle double-free and dangling-pointer bugs.
- Lambdas and STL containers together produce concise, readable move-generation code.
- Value semantics for undo history eliminates a whole class of memory-safety bugs at the cost of trivial extra memory.
- Threading in game loops requires careful design (atomic flags, thread join on reset) but the payoff in user experience is significant.

References

- [1] B. Stroustrup, *The C++ Programming Language*, 4th ed. Addison-Wesley, 2013.
- [2] S. Meyers, *Effective Modern C++*. O'Reilly Media, 2014.
- [3] *cppreference.com*. [Online]. Available: <https://en.cppreference.com>. [Accessed: May 10, 2026].
- [4] Raylib. *raylib – A simple and easy-to-use library to enjoy videogames programming*. [Online]. Available: <https://www.raylib.com>. [Accessed: May 10, 2026].
- [5] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2020. (Chapter 5: Adversarial Search).
- [6] *Chess Programming Wiki – Alpha-Beta*. [Online]. Available: <https://www.chessprogramming.org/Alpha-Beta>. [Accessed: May 10, 2026].

A Complete Source Code

The entire project is contained in a single `main.cpp` file. Key sections are reproduced throughout the report; the complete listing is below.

A.1 main.cpp

```

1  // =====
2  //  CHESS GAME -- C++ OOP + Raylib + Minimax AI Bot
3  //  =====
4  #include "raylib.h"
5  #include <vector>
6  #include <string>
7  #include <stack>
8  #include <stdexcept>
9  #include <algorithm>
10 #include <cstdlib>
11 #include <cstdio>
12 #include <thread>
13 #include <atomic>
14
15 const int CELL = 80;
16 const int BOARD = 8 * CELL;
17 const int PANEL = 240;
18 const int SCR_W = BOARD + PANEL;
19 const int SCR_H = BOARD;
20 const int BOT_DEPTH = 3;
21
22 // Enums, Unicode helpers, InvalidMoveException,
23 // Move, Snapshot, Piece (abstract), Pawn, Knight, Bishop,
24 // Rook, Queen, King, EmptyPiece, Board, AI, Game
25 // ... (see full listing in submission zip) ...
26
27 int main() {
28     InitWindow(SCR_W, SCR_H, "Chess vs Bot - OOP Project");
29     SetTargetFPS(60);
30     // load font, create Game, main loop
31     Game game;
32     while (!WindowShouldClose()) {
33         if (IsMouseButtonPressed(MOUSE_LEFT_BUTTON)) {
34             int mx = GetMouseX(), my = GetMouseY();
35             if (mx < BOARD) game.handleClick(mx, my);
36         }
37         if (IsKeyPressed(KEY_R)) game.reset();
38         if (IsKeyPressed(KEY_U)) game.undoMove();
39         game.botMove();
40         BeginDrawing();
41         ClearBackground({18, 18, 26, 255});
42         game.draw();
43         EndDrawing();
44     }
45     CloseWindow();
46     return 0;
47 }

```

Listing 10: main.cpp – Chess Game vs AI Bot (complete source)

B Installation and Usage

B.1 Prerequisites

- C++17-compatible compiler: g++ 9+ or MSVC 2019+
- Raylib 5.x (download from <https://www.raylib.com>)
- Windows 10/11 (for `seguisym.ttf` chess Unicode font)

B.2 Compilation

```
1 g++ -std=c++17 -Wall -O2 main.cpp ^
2     -I path/to/raylib/include ^
3     -L path/to/raylib/lib ^
4     -lraylib -lopengl32 -lgdi32 -lwinmm ^
5     -o chess
6
7 chess.exe
```

Listing 11: Building with g++ on Windows (MinGW)

B.3 Controls

- **Left Click** – Select a white piece; click a highlighted square to move.
- **U** – Undo the last full move (player + bot).
- **R** – Restart the game.